Progress Report — Jyotir

AMS 560 · Big Data Systems, Algorithms and Networks · Fall 2026 · Group 20

1. Objective and Project Description

We are building a **change-aware, feasibility-first scheduler** for Large Language Model (LLM) inference requests across a cluster of geo-distributed GPU nodes. The core problem is that existing schedulers ignore live hardware state — GPU temperature, power headroom, KV cache fill — and route requests to nodes that are physically unable to serve them. Our system fixes this with a two-stage pipeline: a **feasibility filter** that enforces hard safety rules and eliminates unsafe nodes, followed by a **contextual bandit policy** (sliding-window LinUCB) that learns, from observed outcomes, which feasible nodes produce the best trade-off of latency, goodput, and energy cost over time.

The primary metric is **goodput** — requests per second finishing within the 250 ms time-to-first-token (TTFT) SLO — and **adaptation time** — how quickly goodput returns to ≥90 % of its new steady state after a cluster change event. We evaluate across six scenarios: stable (S1), traffic burst (S2), electricity price shift (S3), thermal throttle (S4), network degradation (S5), and node outage plus recovery (S6). The project is split across five members: Jyotir (filter + baselines), Sur (LinUCB policy + reward), Krupesh (router service + interfaces), Parikshit (node simulator + event traces), and Riya (experiments + metrics + final report).

2. What Has Been Completed

Jyotir — Feasibility Filter and Baselines

The feasibility filter is fully implemented. It enforces five hard rules in order — node status, model availability, KV cache capacity, power headroom, and GPU temperature — and returns a list of allowed nodes plus rejection reasons for every removed node. All thresholds live in a YAML config file so they can be tuned without changing code. A warming-node flag (`_warming=True`) lets downstream policies apply a soft preference penalty without bypassing the safety rules. All six baseline policies are complete: Round-Robin, Min RTT, Least Load, Lowest Energy Cost, Static Weighted, and Power-of-Two. An edge-case test suite of 27 tests covers all six failure categories and passes in full. A stub layer (synthetic node and request objects) lets the filter and baselines run independently without waiting for teammates. The report section covering the filter and baselines is also written and pushed.

Parikshit — Node Simulator and Event Traces

A discrete-event simulator (SimPy) of five GPU nodes is complete, modelling queue depth, KV cache use, power, temperature, RTT, and node status. Latency constants are calibrated to published H100 and A100 benchmarks. Real data loaders pull Azure LLM traces, NYISO/CAISO electricity prices, and inter-city ping data. All six scenario trace files (S1–S6) are written and versioned. The telemetry schema v1 is frozen, including the `kv_capacity_tokens` field needed by the token-exact memory check. 122 tests pass. The simulator section of the report is drafted.

Sur — LinUCB Policy and Reward Function

Both plain LinUCB and sliding-window LinUCB are implemented and tested. The context vector builder produces 14 scaled features per node. The reward function combines SLO compliance, latency, energy cost, and thermal stress with configurable weights in YAML. A window-size sweep over $W \in \{200, 500, 1000, 2000, 5000\}$ on Parikshit's simulator through Krupesh's router identified per-node $W = 500$ as the best default. The policies plug into Krupesh's router via a registry adapter. The method section of the report is drafted.

Krupesh — Router Service and Integration Layer

The routing API contract (request and response fields), decision log format (18-column Parquet schema), and Policy base class interface are defined and frozen. The FastAPI router service with `/route` and `/feedback` endpoints is built. Mock objects for the simulator, policy, and reward function allow end-to-end smoke tests. An integration adapter for Jyotir's filter and a weekly integration check script are in place. The introduction and system design sections of the report are drafted.

Riya — Metrics and Experiment Runner

The metrics module is implemented: goodput, p95/p99 latency, rejection rate, cost per successful request, and adaptation time are all computable from the decision log. The experiment runner accepts a config file specifying policies, scenarios, and seeds, and produces decision logs and summary tables. A config snapshot for the midterm evaluation plan is checked in. Preliminary confidence intervals and a summary table from an initial mock evaluation run are available.

3. What Is Still Left

Baseline decision logs on all six traces (Jyotir). The baselines run correctly against stub data but have not yet been run through the full pipeline (Parikshit's simulator driving Krupesh's router) to produce the Parquet decision logs that Riya needs for every comparison figure. This is the one remaining deliverable from Jyotir.

Real interface swap (Jyotir). The stubs need to be replaced with live imports from Krupesh's Policy base class and Parikshit's telemetry schema. The swap is two lines per baseline file and will happen once the interfaces are confirmed stable.

Full experiment grid (Riya). Riya's runner is built but has not yet produced the full experiment grid (5 policies × 6 scenarios × 10 seeds). Figures (goodput-over-time curves and bar charts with confidence intervals) and the three ablation runs (no filter, no context, no forgetting) are not yet produced.

Final report assembly (Riya). Each member has drafted their own section. Riya has not yet assembled the full report or written the Evaluation, Results, and Discussion sections.

Figures (Riya). No figures have been produced yet; the `figures/` folder is empty. Figures depend on the full experiment grid completing first.

4. Difficulties

Independent development across five modules. The filter, router, simulator, policy, and experiment runner are tightly coupled. Each member needs outputs from other members before certain tasks can complete. This was mitigated with stub layers and a frozen API contract, but it means Jyotir's final deliverable (baseline decision logs) and Riya's figures are both blocked until the end-to-end pipeline is fully wired and verified.

Token-exact memory check. The initial feasibility filter used only a fill-fraction threshold. This was found to be incorrect: a node at 50 % fill with a 1,000-token capacity would pass the fraction check but physically cannot fit a 1,050-token request. The fix required a token-exact check using the `kv_capacity_tokens` field from Parikshit's telemetry schema, with the fraction check kept as a fallback.

Warming node safety gap. An early version of the filter skipped the model, memory, power, and temperature rules for warming nodes, treating status as a pass/fail gate. This meant a warming node that was also out of memory or over temperature would incorrectly be admitted. The filter was restructured so warming nodes go through all five rules and receive the `_warming` tag only if they pass every check.

Price-aware routing with the current reward weights. Sur's window sweep found that with the current reward weights (SLO weight = 1.0, energy weight = 0.1), the learned policy does not shift traffic toward cheaper nodes in Scenario S3. The energy weight is too small relative to the SLO term. Whether to increase the energy weight is an open team decision that must be resolved before the final experiment run.

All of Jyotir's individual code deliverables are complete and tested. The remaining work requires end-to-end pipeline integration with the full team.